

アクセス制御

ファイルやディレクトリには、所有者、所有グループ、アクセス権限が設定されます。

違うユーザに編集されたくないファイルや移動されたくないディレクトリがあれば、所有者、所有グループ、アクセス権の変更を実施することになります。アクセス権(パーミッション)の設定はとても重要です。実務においても必修の知識です。

(1)アクセス権の概要 ←P152

Linux は1つのシステムを複数のユーザで共有できるマルチユーザのシステムですので、システム上のデータをどのユーザも自由に操作できてしまうとセキュリティ上、問題があります。

そこで、ファイルやディレクトリに対してこういった操作を許可するのかというアクセス権を設定します。

アクセス権には読み取り権、書き込み権、実行権の3種類あります。

ファイルやディレクトリに対して、3種類の権限が与えられた場合に可能となる操作を下記にまとめます。

	ファイル	ディレクトリ
読み取り権 r	ファイルの内容を読み取れるが、更新はできない	ディレクトリ内のファイル一覧を表示できる
書き込み権 w	ファイルの内容を変更したり、削除できる	ディレクトリ内でファイルを作成したり、削除できる
実行権 x	ファイルを実行できる	cd コマンドでディレクトリ内に移動できる

※上記の3種類の権限を、ユーザごとに割り当てます。

(2)所有者・所有グループ

一つのLinuxシステムを複数のユーザで使用している場合、自分が作成したファイルを、他のユーザに変更されたくない場合があります。

ファイルには、その持ち主がいて、所有者と呼ばれます。ファイルを作成すると、作成したユーザがその所有者として設定されます。

所有者にだけ書き込み権限を与えておけば、そのファイルを他のユーザに変更される心配はありません。場合によっては、ファイルを、ある特定の人たちの間でだけ共有したい場合もあるでしょう。

ファイルには所有者のほかに、所有グループというものがあるので、所有グループに書き込み権限を与えることで、そのグループに属するユーザはそのファイルを変更することができるようになります。

ファイルを作成すると、作成したユーザのプライマリグループが、その所有グループとして設定されます。

(実習 1)① グループ : **g1** ←P256

```
[hal@kaizuka ~]$ sudo groupadd g1
```

② 以下の条件のユーザ作成 ← **g (P243)を忘れないように！失敗したら下の※参照**ユーザ : **u1** グループ : **g1** パスワード : **halhal**

```
[hal@kaizuka ~]$ sudo useradd u1 -g g1
[hal@kaizuka ~]$ sudo passwd u1
```

③ グループ(/etc/group)、ユーザ(/etc/passwd) が作成されている事の確認

```
[hal@kaizuka ~]$ cat /etc/passwd
```

④ ユーザ : **u1** に切り替えて、ファイル **test.txt** の作成とアクセス権の確認

```
[hal@kaizuka ~]$ su - u1
[u1@kaizuka ~]$ touch test.txt
```

[u1@motobe ~]\$ **ll**

合計 0

-rw-r--r-- 1 u1 g1 0 6月 8 19:56 test.txt

上の例では、グループ「**g1**」をプライマリグループとするユーザ「**u1**」が、touch コマンドでファイル「**test.txt**」を作成したときに、そのファイルの所有者が「**u1**」、所有グループが「**g1**」となっていることが確認できます。

※②を失敗、つまり -g を忘れてグループが **u1** になってしまった時は[hal@motobe ~]\$ **sudo gpasswd -a u1 g1**

←教科書には無し(CB41 Linux 資料 04 ユーザ・グループ管理 で学習済み)

[hal@motobe ~]\$ **cat /etc/group**

で後から変更

(3)所有者の変更 ←P150

ファイルやディレクトリの所有者を変更するには、**chown** コマンドを使用します。このコマンドを使って所有者を変更できるのはスーパーユーザだけです。

所有者を変更すると同時に、所有グループを変更することもできます。

chown [オプション] ユーザ[:グループ] ファイル名/ディレクトリ名・例…[hal@motobe ~]\$**sudo chown hal:group1 test.txt**ファイル **test.txt** の所有者を **hal**、所有グループを **group1** に変更(例です！**group1** はありません)

(実習 2) …作業場所はルート(/)の直下で

① ファイル「aaa.txt」作成

←あらかじめ hal に戻っておくこと

```
[hal@kaizuka ~]$ cd /
[hal@kaizuka /]$ sudo touch aaa.txt
```

② ファイル「aaa.txt」の所有者を「root」から「u1」へ、所有グループを「root」から「g1」へ同時に変更
←上の例

```
[hal@kaizuka /]$ sudo chown u1:g1 aaa.txt
```

[hal@motobe /]\$ ll /aaa.txt

-rw-r--r-- 1 u1 g1 0 6月 8 20:01 /aaa.txt

③ ディレクトリ「dir」の作成とその中にファイル「bbb.txt」作成

```
[hal@kaizuka /]$ sudo mkdir dir
[hal@kaizuka /]$ sudo touch dir/bbb.txt
```

④ディレクトリ「dir」および配下の全ファイル（下の例の場合は「bbb.txt」）の所有者を「root」から「u1」へ変更 ←-R を忘れないように！

[hal@motobe /]\$ ls -ld dir/

[hal@motobe /]\$ **sudo chown -R u1 dir** ←P147

⑤変更確認

[hal@motobe /]\$ ls -ld dir/

drwxr-xr-x 2 **u1** root 21 6月 8 20:06 dir/

[hal@motobe /]\$ ls -ld dir/bbb.txt

-rw-r--r-- 1 u1 root 0 6月 8 20:06 dir/bbb.txt**(4)所有グループの変更** ←P150

ファイルやディレクトリの所有グループを変更するには、chgrp コマンドを使用します。

(実習 3)① グループ：**g2** 作成

```
[hal@kaizuka /]$ sudo groupadd g2
```

② ファイル「aaa.txt」の所有グループを「**g1**」から「**g2**」に変更[hal@motobe /]\$ **sudo chgrp g2 aaa.txt**

[hal@motobe /]\$ ll -l aaa.txt

-rw-r--r-- 1 u1 g2 0 6月 8 20:01 aaa.txt

※このコマンドは一般ユーザでも使用できますが、制限があります。

一般ユーザはファイル（ディレクトリ）の所有グループを、自分が所属するグループから、自分が所属する別のグループに変更することだけ許可されています。

言い換えると、ファイル（ディレクトリ）のもともとの所有グループが、自分が所属しないグループである場合、そのファイル（ディレクトリ）の所有グループを変更することはできません。

また、ファイル（ディレクトリ）のもともとの所有グループが、自分が所属するグループである場合でも、そのファイル（ディレクトリ）の所有グループを、自分が所属しないグループに変更することはできません。

(5)アクセス権

←P152

ファイルやディレクトリにはアクセス権が設定できます。

アクセス権とはパーミッションとも呼ばれ、どのユーザに、どの操作を許可するのかという情報のことです。

まず、「どのユーザに」の部分ですが、Linux ではユーザを下に示す 3 種類に分類します。

所有者

所有グループに属するユーザ

その他のユーザ

次に、「どの操作を」の部分ですが、これもまた 3 種類あります。

読み取り可能 **readable**

書き込み可能 **writable**

実行可能 **executable**

(6)アクセス権の数値表記

「(5)アクセス権」では、3 種類のユーザに対して 3 種類の権限を設定すること、また、おのそのの権限は「**rwX**」などといった文字列で表現できることを紹介しました。

ここでは数値によるアクセス権の表記方法を紹介します。

数値によるアクセス権の表記では、下表のように権限を数値で表します。

権限	記号表記	数値表記
読み取り権限	r	4
書き込み権限	w	2
実行権限	x	1

そして、3 種類のユーザごとに、与えられた権限を表す数値の和をとります。

例えば、所有者に与えられた権限が「**rwX**」の場合、「4」+「2」+「1」=「7」となります。

(問題) アクセス権の記号表記が「**rwXr—Xr—**」のとき、数値表記は幾つか？

(7)アクセス権の変更

ファイルやディレクトリのアクセス権を変更するには、**chmod** コマンドを使用します。

chmod [オプション] アクセス権ファイル名/ディレクトリ名

アクセス権の指定は、「対象ユーザ」、「操作」、「権限」の3つを指定します。

(実習 4)

① 現状確認

```
[ ]
```

② ファイル「aaa.txt」に対して、その他のユーザ「o」に、書き込み権限「w」を追加「±」

```
[hal@motobe ~]$ sudo chmod o+w aaa.txt
```

③ ファイル「aaa.txt」に対して、所有者「u」および所有グループに属するユーザ「g」から、読み取り権限「r」を削除「-」

```
[ ]
```

```
[hal@motobe ~]$ ll -l aaa.txt
```

```
--w----rw- 1 u1 g2 0 6月 8 20:01 aaa.txt
```

(8)アクセス権の数値による変更

「(6)アクセス権の数値表記」にあるように、アクセス権は数値で表現することができ、chmod コマンドの引数に変更したいアクセス権の数値を直接指定することで、そのアクセス権に変更できます。

(実習 5)

①現状確認

③ ファイル「aaa.txt」のアクセス権を「777」(rw-rw-rw-)に変更

```
[hal@motobe ~]$ sudo chmod 777 aaa.txt
```

※ アクセス権の数値による変更では、記号による変更とは異なり、特定のユーザに特定の権限を追加するといった指定の仕方はできません。例えば「所有者」に「実行権」を追加する場合は「**chmodu+x test.txt**」のように記号を使います。

(9)SUID ←教科書になし

SUID (Set User ID) とは、特殊なパーミッション (アクセス権) の一つで、実行可能ファイルに対して設定できます。

`/etc/passwd` ファイルはユーザ名やパスワードなどのユーザアカウント情報を格納するファイルです。このファイルはパーミッションが「`rw-r--r--`」となっていて、所有者である「**root**」以外は書き込みができません。

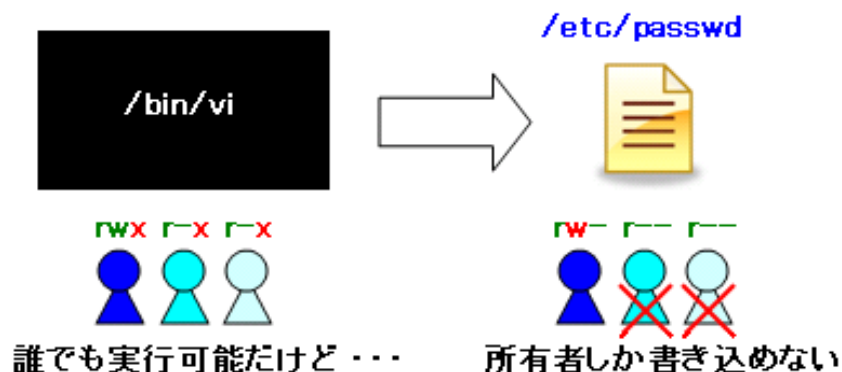
例えば、`vi` のようなテキストエディタはパーミッションが「`rw-r--r--`」となっていて誰でも実行 (起動) 可能ですが、`vi` で `/etc/passwd` ファイルを直接編集しようとした場合、ファイルのほうのパーミッションによって、**一般ユーザは書き込みができません。**

① アクセス権を確認

```
[hal@motobe /]$ ls -l /etc/passwd
-rw-r--r-- 1 root root 2679  6月  8 19:52 /etc/passwd
```

```
[hal@motobe /]$ ls -l /bin/vi
-rwxr-xr-x. 1 root root 691  2月 10 01:36 /bin/vi
```

②



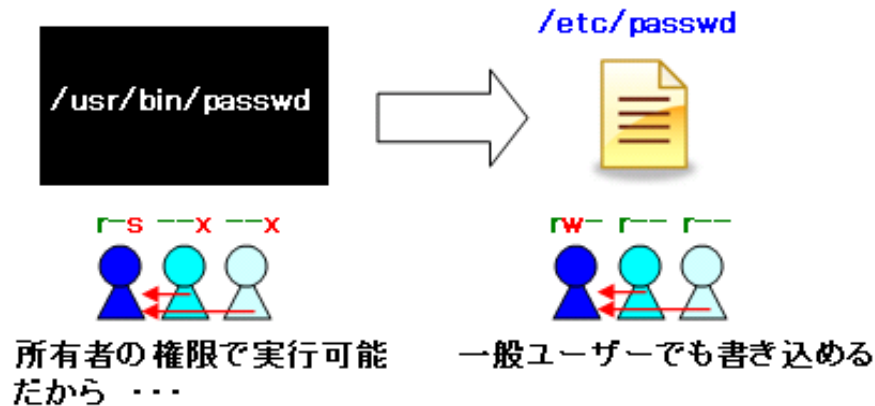
しかし、一般ユーザが `passwd` コマンドで自身のパスワードを変更すると、新しいパスワードは `/etc/passwd` ファイルに保存されます。

つまり、一般ユーザが `/etc/passwd` ファイルに書き込めたことになります。

同じ一般ユーザで実行しているのに、`/etc/passwd` ファイルに書き込めなかった `vi` エディタと、書き込めた `passwd` コマンドの違いは何でしょうか？

それは、`passwd` コマンドには特殊なパーミッションである **SUID** が設定されているということです。

SUID が設定されたコマンド (ファイル) を実行すると、**ファイルの所有者の権限で実行**されます。 `passwd` コマンドには **SUID** が設定されているので、誰が実行したときでも所有者である「**root**」の権限で実行され (所有者である「**root**」の User ID が Set され)、「**root**」なので **`/etc/passwd` ファイルに書き込むことができる**わけです。



SUID を設定するには、chmod コマンドで所有者「u」に対して権限「s」を追加します。 ←設定済

```
[hal@motobe /]$ ls -l /usr/bin/passwd
```

```
-rwsr-xr-x. 1 root root 32648  8月 10  2021 /usr/bin/passwd
```

※ /etc/passwd ファイルは vi で自由に編集されると、書式などが崩れ、ユーザがログインできなくなる恐れがあります。一方、passwd コマンドは書式を保ち、適切に編集してくれます。編集対象となる ファイルのパミッションを緩める（全てのユーザに書き込み権限を与える）ことなく、特定のプログラム（コマンド）からのみ編集することができるようにする仕組みが SUID です。

※余計な SUID はセキュリティホールの原因になります。

※SUID の設定しているファイルの検索

```
[hal@motobe /]$ find /usr/bin -perm -u+s
```

(実習 6)

ユーザ u1 で/etc/shadow ファイル(パスワード設定)を確認できるようにする

① パスワードファイルのアクセス権の確認

```
[hal@motobe /]$ ls -l /etc/shadow
```

② cat コマンド(/bin/cat)に SUID を設定、確認

```
[hal@motobe /]$ sudo chmod u+s /bin/cat
```

←cat コマンドに SUID を付加

```
[hal@motobe /]$ ls -l /bin/cat
```

※この様に cat コマンドに不用意に SUID を付加するとセキュリティを下げますので元に戻しておくこと！

④ セキュリティの為に元に戻す(SUID を外す)

```
[hal@motobe /]$ sudo chmod u-s /bin/cat
```

←cat コマンドから SUID を削除。「-」はマイナスです

以下の(10)~(11)は時間の有る学生用・・・必須では無い

(10)SGID ←教科書になし

SGID (Set Group ID) とは、特殊なパーミッション (アクセス権) の一つで、実行可能ファイルおよびディレクトリに対して設定できます。

①まず、実行可能ファイルに設定する場合からみてみましょう。

実行権限を持っているユーザによって、SUID の設定されたファイルが実行された場合、**ファイルの所有者の権限で実行**されました。

同様に、SGID が設定されたファイルが実行された場合は、**ファイルの所有グループの権限で実行**されます。

②次に、ディレクトリに SGID を設定する場合をみてみましょう。

ディレクトリに対して SGID を設定すると、そのディレクトリ内に作成されたファイルやディレクトリの所有グループが、ディレクトリ自体の所有グループに設定されます。

ディレクトリに設定する SGID は複数ユーザで共同作業をする場合に便利です。

例えばプライマリグループが「**G1**」のユーザ「**U1**」と、プライマリグループが「**G2**」のユーザ「**U2**」がいるとします。

この2人のユーザが共同作業用のディレクトリに、それぞれファイルを作成した場合、お互い、相手のファイルには書き込むことができません。ユーザ「**U1**」が作成したファイルに書き込めるのは所有者、つまり「**U1**」か、所有グループ、つまり「**G1**」に属するユーザだけです。ユーザ「**U2**」はグループ「**G1**」には属していないから、**書き込めない**わけです。



ここで共同作業用のディレクトリに SGID を設定してみましょう。共同作業用のディレクトリの所有グループを「**G3**」とします。そしてユーザ「**U1**」とユーザ「**U2**」をグループ「**G3**」に所属させます。

この状態で、2人のユーザがそれぞれファイルを作成すると、作成されたファイルの所有グループが「**G3**」となるので、**お互い、相手のファイルに書き込むことができる**ようになります。



SGID を設定するには、**chmod** コマンドで所有グループ「**g**」に対して権限「**s**」を追加します。

(実習 7)

- ① ディレクトリ「**dir**」の作成
 - ② 所有グループ「**g**」に対して権限「**s**」を追加
- ```
[hal@motobe /]$sudo chmod g+s dir
```
- ```
[hal@motobe /]$ls -ld dir
```

(実習 8)…作業場所はルート(/)の直下で

- ① (10)の説明を実際に確認
 - ・ユーザ : 「**u1**」、「**u2**」
 - ・グループ : 「**g1**」、「**g2**」、「**g3**」 ← 「**g3**」を新たに作成
 - ・ディレクトリ「**work**」の作成 ← パーミッションを「**777**」する
 - ・ディレクトリ「**work**」内に「**u1**」がファイル「**u1.txt**」、「**u2**」がファイル「**u2.txt**」を作成
← ユーザを切り替えて作成
- ② まず、お互いに書き込めない事の確認
← ユーザ **u1** は **u2.txt** に書き込めない。ユーザ **u2** は **u1.txt** に書き込めない。ユーザを切り替えて確認。
- ③ ユーザ「**u1**」とユーザ「**u2**」をグループ「**g3**」に所属 ← CB41 資料 06 ユーザ・グループ管理 参照
- ④ **u1.txt** と **u2.txt** を削除
- ⑤ ディレクトリ「**work**」のグループを「**g3**」に、また権限「**s**」を追加
- ⑥ もう一度、ディレクトリ「**work**」内にユーザ「**u1**」が **u1.txt** 作成(**g+w** も追加)、ユーザ「**u2**」が **u2.txt** を作成(**g+w** も追加)
- ⑦ **u1.txt** と **u2.txt** のグループが「**g3**」になっていることの確認

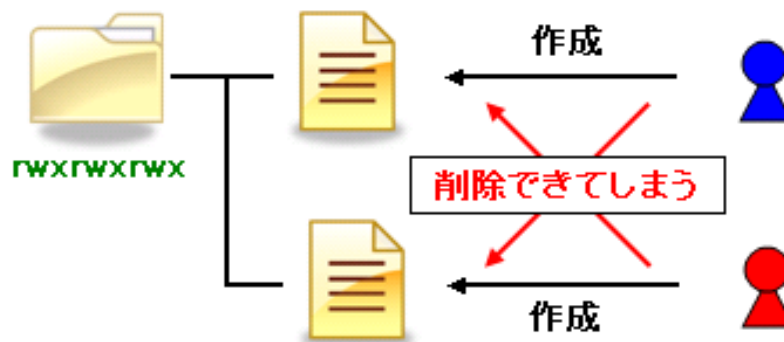
⑧お互い書き込めることの確認

※それぞれのユーザがファイルを作成するたびに、そのファイルの所有グループを「g3」に変更してもよいですが、SGIDはその手間を省いてくれます。←SGIDの有るディレクトリ内では自動で同じグループが付く。

(11)スティッキービット 時間の有る学生用・・・必須では無い ←教科書になし

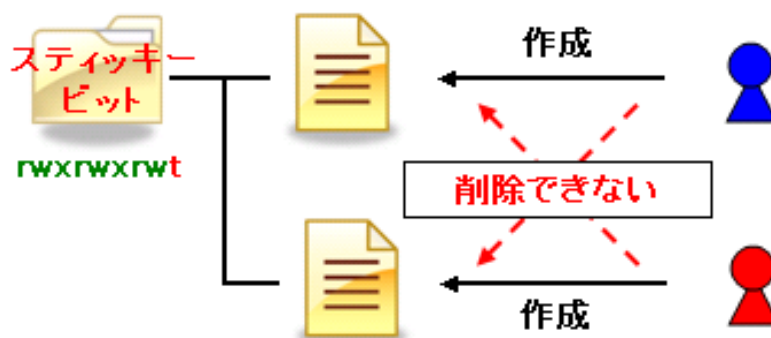
スティッキービットとは特殊なパーミッション（アクセス権）の一つで、**ディレクトリに対して設定**できます。

誰でも自由に書き込みができるディレクトリを作成する場合、そのディレクトリのパーミッションを「rwxrwxrwx」（777）にすればよいですが、誰もが自由に書き込みができるということは、そのディレクトリの中に作成されたファイルを、**誰もが自由に削除できる**ことを意味します。



自分が作成したファイルが、勝手に削除されては困りますね。

そこで、このディレクトリに対してスティッキービットを設定します。スティッキービットが設定されたディレクトリでは、書き込み権限はあっても、自分以外のユーザが所有するファイルを削除することができません。



スティッキービットを設定するには、`chmod` コマンドでその他のユーザ「o」に対して権限「t」を追加

(実習 9)

①前準備

- ・ `dir` ディレクトリのアクセス権を `777` に設定
- ・ 変更確認

④ スティッキービットの設定

```
[hal@motobe /]$ sudo chmod o+t dir
```

```
[hal@motobe /]$ ls -ld dir/
```

```
drwxr-xr-t 2 u1 root 21  6月  8 20:06 dir/
```

※その他のユーザ(o)の実行権が「t」になる

(実習 10)

① 上記の説明(11)を実際に確認

- ・ ディレクトリ「`work2`」の作成
- ・ ディレクトリ「`work2`」内に「ユーザ `u1`」がファイル「`u3.txt`」、「ユーザ `u2`」がファイル「`u4.txt`」を作成

② 「`u3.txt`」、「`u4.txt`」のアクセス権を `777` にして「`u1`」でアクセス

←「`u1`」は「`u4.txt`」の書き込み、削除も出来る

③ ディレクトリ「`work2`」にスティッキービットを設定

⑤ 「`u1`」は「`u4.txt`」を**書き込めるが削除できない事**の確認

以上！